

FutureKawa

Documentation Backend — Exploitation

Équipe : Rudy · Florian · Noé · Josselin

Version : 1.0

Date : 05/07/2026

Projet : FutureKawa

Sommaire

Backend Exploitation — FutureKawa

01 — Vue d'ensemble

02 — Django & Django REST Framework

03 — Endpoints de l'API

04 — Celery & Redis

05 — Temps réel MQTT

06 — Emails & notifications

07 — Configuration & déploiement

08 — Tests

Backend Exploitation — FutureKawa

Projet	FutureKawa
Livrable	Documentation Backend Exploitation
Périmètre	API Django de l'instance exploitation (capteurs, lots, alertes, incidents, temps réel)
Framework	Django 4.2 + Django REST Framework
Asynchrone	Celery 5 + Redis + Celery Beat
Temps réel	Broker MQTT (paho-mqtt)
Équipe	Rudy · Florian · Noé · Josselin

Sommaire

Section	Fichier
Vue d'ensemble	01-vue-ensemble.md
Django & Django REST Framework	02-django-drf.md
Endpoints de l'API	03-endpoints-api.md
Celery & Redis	04-celery-redis.md
Temps réel MQTT	05-mqtt-temps-reel.md
Emails & notifications	06-emails-notifications.md
Configuration & déploiement	07-configuration.md
Tests	08-tests.md

Composants du service

Composant	Rôle	Technologie
API REST	Expose les données aux frontends	Django REST Framework
Subscriber MQTT	Ingère les mesures des capteurs	paho-mqtt
Worker Celery	Exécute les tâches asynchrones (emails, alertes)	Celery
Celery Beat	Planifie les tâches périodiques	django-celery-beat
Redis	File d'attente des tâches Celery	Redis
Endpoint <code>/api/all-data/</code>	Alimente le siège (master token)	DRF

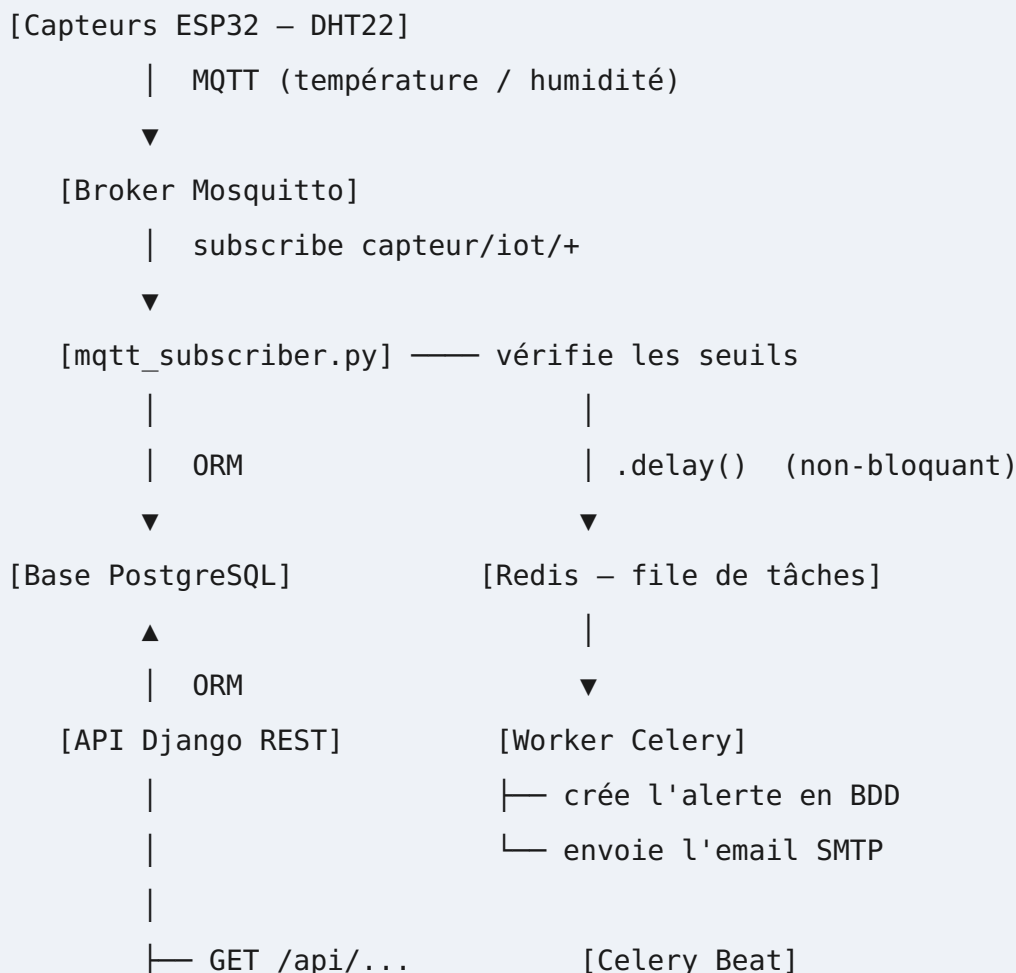
01 — Vue d'ensemble

Rôle du backend exploitation

Le backend exploitation est le **cœur logique d'une plantation**. Chaque plantation déploie sa propre instance, indépendante des autres et du siège. Il assure quatre missions :

1. **Ingérer** les mesures des capteurs IoT en temps réel (via MQTT).
2. **Stocker** ces mesures et toutes les données métier (lots, entrepôts, alertes, incidents) en PostgreSQL.
3. **Réagir** automatiquement : détection de seuils dépassés, création d'alertes, envoi d'emails.
4. **Exposer** les données au frontend exploitation et au siège (endpoint `/api/all-data/`).

Architecture du service



```
| (front exploitation) └─ check_stale_sensors (toutes les  
15 s)  
|  
└─ GET /api/all-data/ → [API Siège] (master token)
```

Pourquoi trois processus ?

Le service ne se résume pas à « une API ». Il tourne en **trois processus distincts** qui partagent le même code et la même base :

Processus	Commande	Rôle
Web	<code>gunicorn core.wsgi</code>	Sert l'API REST
MQTT	<code>python mqtt_subscriber.py</code>	Écoute les capteurs
Worker	<code>celery -A core worker</code>	Exécute les tâches asynchrones
Beat	<code>celery -A core beat</code>	Déclenche les tâches planifiées

Ce découpage est ce qui permet à l'API de rester rapide : elle ne bloque jamais sur un envoi d'email ou un traitement long. Voir [Celery & Redis](#).

Stack technique

Élément	Choix	Rôle
Framework web	Django 4.2	Structure, ORM, migrations
API	Django REST Framework	Sérialisation, ViewSets, routeurs
Base de données	PostgreSQL 15	Persistance
File de tâches	Redis	Broker Celery
Worker async	Celery 5	Emails, alertes

Élément	Choix	Rôle
Planificateur	django-celery-beat	Tâches périodiques
Temps réel	paho-mqtt	Subscriber MQTT
Documentation API	drf-spectacular	Swagger / OpenAPI

Dépendances (requirements.txt)

```
Django>=4.2,<5.0
djangorestframework>=3.14
psycopg2-binary>=2.9      # driver PostgreSQL
django-cors-headers>=4.0   # CORS pour le front
python-dotenv>=1.0.0       # variables d'environnement
paho-mqtt>=1.6.1           # client MQTT
drf-spectacular>=0.26.0    # documentation OpenAPI
celery>=5.3.0              # tâches asynchrones
redis>=4.5.0               # broker Celery
django-celery-beat>=2.5.0  # tâches planifiées
```

02 — Django & Django REST Framework

Qu'est-ce que Django ?

Django est un framework web Python « batteries incluses » : il fournit d'office tout ce qu'il faut pour construire une application web robuste sans réinventer la roue.

Django suit le principe **MVT** (Model – View – Template), une variante du MVC : - **Model** : la structure des données (une classe Python = une table SQL). - **View** : la logique qui répond à une requête. - **Template** : la présentation (peu utilisé ici, car le rendu se fait côté Vue.js).

Ce que Django apporte au projet

Fonctionnalité	À quoi ça sert dans FutureKawa
ORM	On manipule des objets Python, Django génère le SQL PostgreSQL
Migrations	Le schéma de la base est versionné dans le code (<code>migrations/</code>)
Système d'authentification	Comptes utilisateurs, groupes, permissions
Admin	Interface d'administration auto-générée
Gestion des settings	Configuration par environnement (local / prod / test)
Commandes de gestion	<code>python manage.py ...</code> (migrations, seed, etc.)

L'ORM : une classe = une table

Au lieu d'écrire du SQL, on décrit les données comme des classes. Django s'occupe de créer et requêter les tables.

```
# exploitation/models.py
class Warehouse(models.Model):
    name = models.CharField(max_length=100)
    location = models.CharField(max_length=200)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'warehouse'
```

Requêter devient du Python lisible :

```
Warehouse.objects.all()           # SELECT * FROM
warehouse
Warehouse.objects.filter(location='Bogota') # ... WHERE location =
'Bogota'
Warehouse.objects.get(id=3)        # ... WHERE id = 3
```

Le détail complet des tables est dans le livrable **Base de données Exploitation**. Ici, on documente comment le backend s'organise **autour** de ces modèles.

Les migrations

Chaque changement de modèle produit un fichier de migration, appliqué avec `migrate` :

```
python manage.py makemigrations # génère le fichier de migration
python manage.py migrate        # applique les changements en base
```

Avantage : le schéma est **reproductible** sur chaque plantation, sans script SQL manuel.

Django REST Framework (DRF)

Django seul renvoie du HTML. Pour exposer une **API JSON** consommée par Vue.js, on ajoute **Django REST Framework**.

DRF s'articule autour de trois briques :

1. Serializer — traduire objet ↔ JSON

```
class WarehouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = Warehouse
        fields = ['id', 'name', 'location', 'created_at']
```

Le serializer valide les données entrantes et convertit les objets en JSON sortant.

2. ViewSet — regrouper les actions CRUD

```
class WarehouseViewSet(viewsets.ModelViewSet):
    queryset = Warehouse.objects.all()
    serializer_class = WarehouseSerializer
    permission_classes = [IsAuthenticated]
```

Un `ModelViewSet` fournit automatiquement `list`, `create`, `retrieve`, `update`, `destroy`.

3. Router — générer les URLs

```
router = DefaultRouter()
router.register(r'entrepots', WarehouseViewSet, basename='warehouse')
```

Génère automatiquement :

Méthode	URL	Action
GET	/api/entrepots/	Liste
POST	/api/entrepots/	Création
GET	/api/entrepots/{id}/	Détail

Méthode	URL	Action
PUT/PATCH	/api/entrepots/{id}/	Mise à jour
DELETE	/api/entrepots/{id}/	Suppression

Configuration DRF du projet

```
# core/settings/base.py
REST_FRAMEWORK = {
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
    # authentication par token, permissions par défaut, etc.
}
```

Réglage	Rôle
Authentification par token	Chaque requête porte un <code>Authorization: Token ...</code>
<code>drf-spectacular</code>	Génère la doc OpenAPI (Swagger)
<code>corsheaders</code>	Autorise le front (autre origine) à appeler l'API

Organisation des settings

Le projet n'a pas un seul `settings.py` mais un **package** par environnement :

```
core/settings/
├─ base.py          # configuration commune
├─ local.py         # développement (DEBUG=True)
├─ production.py    # production (DEBUG=False, CORS restreint)
└─ test.py          # tests (Celery en mode synchrone, SQLite)
```

On choisit l'environnement via

```
DJANGO_SETTINGS_MODULE=core.settings.local .
```

En test, `CELERY_TASK_ALWAYS_EAGER = True` : les tâches Celery s'exécutent immédiatement, sans Redis — pratique pour tester sans infrastructure.

Documentation automatique de l'API

Grâce à `drf-spectacular`, trois URLs sont disponibles :

URL	Contenu
<code>/api/schema/</code>	Schéma OpenAPI brut (YAML)
<code>/api/docs/</code>	Interface Swagger interactive
<code>/api/redoc/</code>	Documentation ReDoc

03 — Endpoints de l'API

Point d'entrée

Toutes les routes sont montées sous `/api/` (`core/urls.py` → `exploitation.api.urls`).

```
urlpatterns = [
    path('', home_view, name='home'),
    path('api/', include('exploitation.api.urls')),
    path('api/schema/', SpectacularAPIView.as_view()),
    path('api/docs/', SpectacularSwaggerView.as_view()),
    path('api/redoc/', SpectacularRedocView.as_view()),
]
```

Ressources CRUD (ViewSet + routeur)

Ces ressources exposent automatiquement les cinq actions REST (liste, création, détail, mise à jour, suppression) :

Route	ViewSet	Ressource
<code>/api/batches/</code>	<code>BatchViewSet</code>	Lots de café
<code>/api/lots/</code>	<code>LotViewSet</code>	Lots (vue métier)
<code>/api/alerts/</code>	<code>AlertViewSet</code>	Alertes
<code>/api/entrepots/</code>	<code>WarehouseViewSet</code>	Entrepôts
<code>/api/zones/</code>	<code>ZoneViewSet</code>	Zones / parcelles
<code>/api/sensors/</code>	<code>SensorViewSet</code>	Capteurs
<code>/api/users/</code>	<code>UserViewSet</code>	Utilisateurs
<code>/api/incidents/</code>	<code>IncidentViewSet</code>	Incidents
<code>/api/incident-types/</code>	<code>IncidentTypeViewSet</code>	Types d'incidents

Endpoints fonctionnels (vues dédiées)

Route	Méthode	Rôle
/api/health/	GET	Sonde de santé (monitoring / CI)
/api/hello/	GET	Test de connectivité
/api/auth/login/	POST	Connexion, renvoie un token
/api/auth/set-password/	POST	Définition du mot de passe via token d'invitation
/api/parameters/	GET/PUT	Paramètres globaux de l'exploitation
/api/page-permissions/	GET	Permissions d'accès par page/rôle
/api/lots/{uuid}/history/	GET	Historique des mouvements d'un lot
/api/sensors/{id}/history/	GET	Historique des mesures d'un capteur
/api/capteur/latest/	GET	Dernières mesures des capteurs
/api/capteur/history/	GET	Historique des mesures
/api/capteur/stats/	GET	Statistiques agrégées (min/max/moyenne)
/api/all/	GET	Vue globale consolidée

Endpoint inter-service : /api/all-data/

C'est le **pont vers le siège**. Le siège appelle cet endpoint pour récupérer l'intégralité des données de la plantation.

```
GET /api/all-data/
```

```
Authorization: Token <MASTER_TOKEN>
```

- Protégé par un **master token** statique (distinct des sessions utilisateurs).
- Renvoie en une seule réponse : entrepôts, zones, capteurs, lots, mesures, alertes, incidents.
- Le siège n'a **jamais** d'accès direct à la base de l'exploitation — uniquement via cet endpoint.

```
[API Siège] —GET /api/all-data/ (master token)→ [API Exploitation]
                                     |
                                     ▼
                                     agrège toute la BDD
```

Authentification

Type de jeton	Usage	En-tête
JWT utilisateur	Émis à la connexion (/ api/auth/login/), signé HMAC-SHA256, valable 24 h	Authorization: Bearer <jwt>
Token d'invitation	À usage unique, permet à un nouvel utilisateur de définir son mot de passe	—
Master token	Statique, réservé à l'accès siège → /api/ all-data/	Authorization: Token <master>

Chaque requête utilisateur authentifiée porte l'en-tête (ou le cookie HttpOnly `session_token`) :

```
Authorization: Bearer eyJhbGciOi...
```

Détail complet dans le livrable [Sécurité — Authentification et API — Authentification](#).

Exemple de réponse

```
GET /api/capteur/latest/  
[  
  {  
    "sensor_id": 12,  
    "warehouse": "Entrepôt Nord",  
    "temperature": 24.7,  
    "humidity": 61.2,  
    "measured_at": "2026-07-05T14:32:10Z"  
  }  
]
```


04 — Celery & Redis

Cette section est une synthèse dans le contexte du backend.

Chaque technologie dispose aussi d'un livrable complet et autonome : [Redis](#) et [Celery](#).

Pourquoi deux services de plus pour envoyer un email ?

Le problème sans Redis/Celery

Broker MQTT

|



mqtt_subscriber.py

- ├─ 1. Reçoit une mesure de température
- ├─ 2. Vérifie les seuils
- ├─ 3. Envoie l'email (SMTP – 2 à 5 secondes) ← BLOQUANT
- ├─ 4. Crée l'alerte en base
- └─ 5. Seulement là → mesure suivante

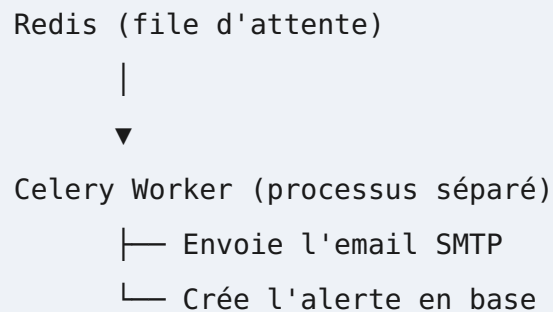
Conséquences :

- Pendant l'envoi de l'email, **aucun capteur n'est écouté**.
- Si le serveur SMTP est lent → des messages MQTT sont perdus.
- Si Gmail est indisponible → le subscriber plante.

La solution : découpler avec Redis + Celery

mqtt_subscriber.py

- ├─ 1. Reçoit une mesure
- ├─ 2. Vérifie les seuils
- ├─ 3. Pousse une tâche dans Redis (< 1 ms) ← NON-BLOQUANT
- └─ 4. Immédiatement prêt pour la mesure suivante



Redis — la file de messages

Redis = base de données **en mémoire**, ultra-rapide, utilisée ici comme **file de tâches** (message broker).

Rôle	Description	Réglage
Broker	Stocke les tâches en attente	<code>CELERY_BROKER_URL</code>
Backend de résultats	Stocke le résultat des tâches	<code>CELERY_RESULT_BACKEND</code>
Port	Écoute réseau	<code>6379</code>

```
# core/settings/base.py
CELERY_BROKER_URL = os.getenv('CELERY_BROKER_URL', 'redis://redis:6379/0')
CELERY_RESULT_BACKEND = os.getenv('CELERY_RESULT_BACKEND', 'redis://redis:6379/0')
```

Redis ne **traite** pas les tâches : il les **stocke** en attendant qu'un worker les prenne.

Celery — le worker asynchrone

Celery = processus Python qui **consomme** les tâches depuis Redis et les **exécute**.

Configuration (`core/celery.py`)

```
app = Celery('core')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks() # trouve automatiquement exploitation/
tasks.py
```

Réglages Celery du projet

```
CELERY_ACCEPT_CONTENT = ['application/json']
CELERY_TASK_SERIALIZER = 'json'
CELERY_TIMEZONE = 'Europe/Paris'
CELERY_TASK_TIME_LIMIT = 30 * 60 # 30 min max par tâche
CELERY_TASK_SOFT_TIME_LIMIT = 25 * 60
```

Tâches définies (`exploitation/tasks.py`)

Tâche	Rôle
<code>create_alert</code>	Crée une alerte en base PostgreSQL
<code>send Consolidated_alert_email</code>	Email HTML groupé (lots affectés + seuils)
<code>send_sensor_alert_email</code>	Email d'alerte capteur
<code>send_incident_notification</code>	Email de notification d'incident manuel
<code>check_stale_sensors</code>	Détecte les capteurs muets (tâche planifiée)

```
# Appel non-bloquant depuis mqtt_subscriber.py
send Consolidated_alert_email.delay(
    warehouse_id, alert_type, batch_ids, value, 'temperature',
    threshold
)
# .delay() = « mets ça dans Redis, ne t'occupe pas du reste »
```

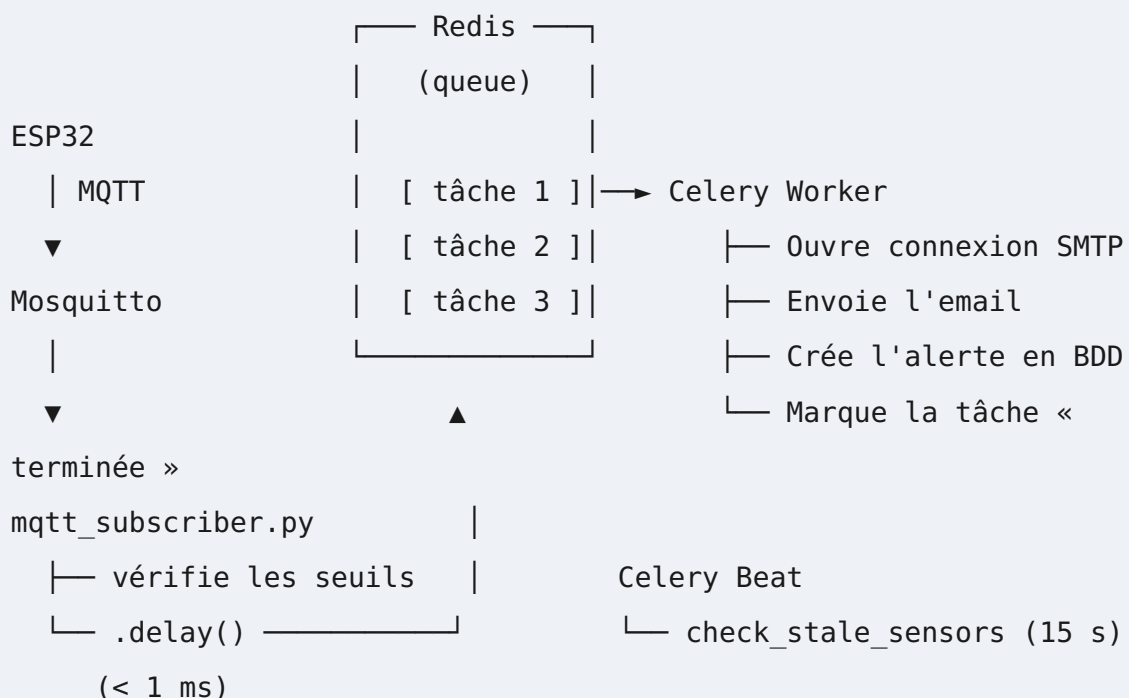
Celery Beat — les tâches planifiées

Celery Beat est un planificateur : il pousse des tâches dans Redis à intervalle régulier, sans qu'un événement extérieur ne les déclenche.

```
# core/settings/base.py
CELERY_BEAT_SCHEDULE = {
    'check-stale-sensors': {
        'task': 'exploitation.tasks.check_stale_sensors',
        'schedule': 15.0,  # toutes les 15 secondes
    },
}
```

Ici, toutes les 15 secondes, Beat déclenche `check_stale_sensors` qui repère les capteurs n'ayant plus émis de mesure (panne, coupure réseau) et lève une alerte.

Le flux complet



Le subscriber ne sait jamais si l'email a réussi. C'est voulu : si Gmail est indisponible, **Celery retente automatiquement**.

Résilience — gestion des erreurs

```
@shared_task(bind=True, max_retries=3)
def send_consolidated_alert_email(self, ...):
    try:
        ... # envoie l'email
    except Exception as exc:
        raise self.retry(exc=exc, countdown=60)
        # Celery remet la tâche dans Redis, réessai dans 60 s
```

Scénario	Comportement
Gmail temporairement indisponible	Réessai ×3 toutes les 60 s
Redis redémarre	Les tâches en attente sont conservées
Worker Celery planté	Les tâches restent dans Redis jusqu'au redémarrage

Anti-spam — cooldown

Sans protection, chaque mesure hors seuil enverrait un email. Un cooldown limite la fréquence :

- Alerte **critique** : 1 email max toutes les **30 minutes** par entrepôt.
- Alerte **warning** : 1 email max toutes les **60 minutes** par entrepôt.

Résumé

```
Redis   = La boîte aux lettres – reçoit et garde les demandes au chaud
Celery  = Le facteur – prend les lettres et les livre
Beat    = L'horloge – dépose une lettre à intervalle régulier
.delay() = Déposer une lettre dans la boîte (non-bloquant)
```

	Redis	Celery	Celery Beat
Rôle	File de messages	Worker de tâches	Planificateur
Technologie	Store en mémoire	Worker async Python	Scheduler

	Redis	Celery	Celery Beat
Port	6379	—	—
Si down	Tâches perdues	Tâches en attente dans Redis	Plus de tâches planifiées

Concept	Description
---------	-------------

Wildcard

+ remplace un niveau :
 capteur/iot/+ = tous les capteurs

MQTT est **léger** et conçu pour l'IoT : idéal pour des milliers de petits messages, là où HTTP serait trop lourd.

Fonctionnement du subscriber

```
# mqtt_subscriber.py (extrait simplifié)
import paho.mqtt.client as mqtt

def on_message(client, userdata, msg):
    payload = json.loads(msg.payload)      # {"temperature": 24.7,
    "humidity": 61}
    # 1. Enregistrer la mesure en base (ORM Django)
    Measure.objects.create(sensor=sensor, temperature=...,
    humidity=...)
    # 2. Vérifier les seuils
    if temperature > threshold:
        # 3. Déléguer l'email/alerte à Celery – NON-BLOQUANT
        send Consolidated_alert_email.delay(...)

client = mqtt.Client()
client.on_message = on_message
client.subscribe("capteur/iot/+")          # tous les capteurs
client.loop_forever()
```

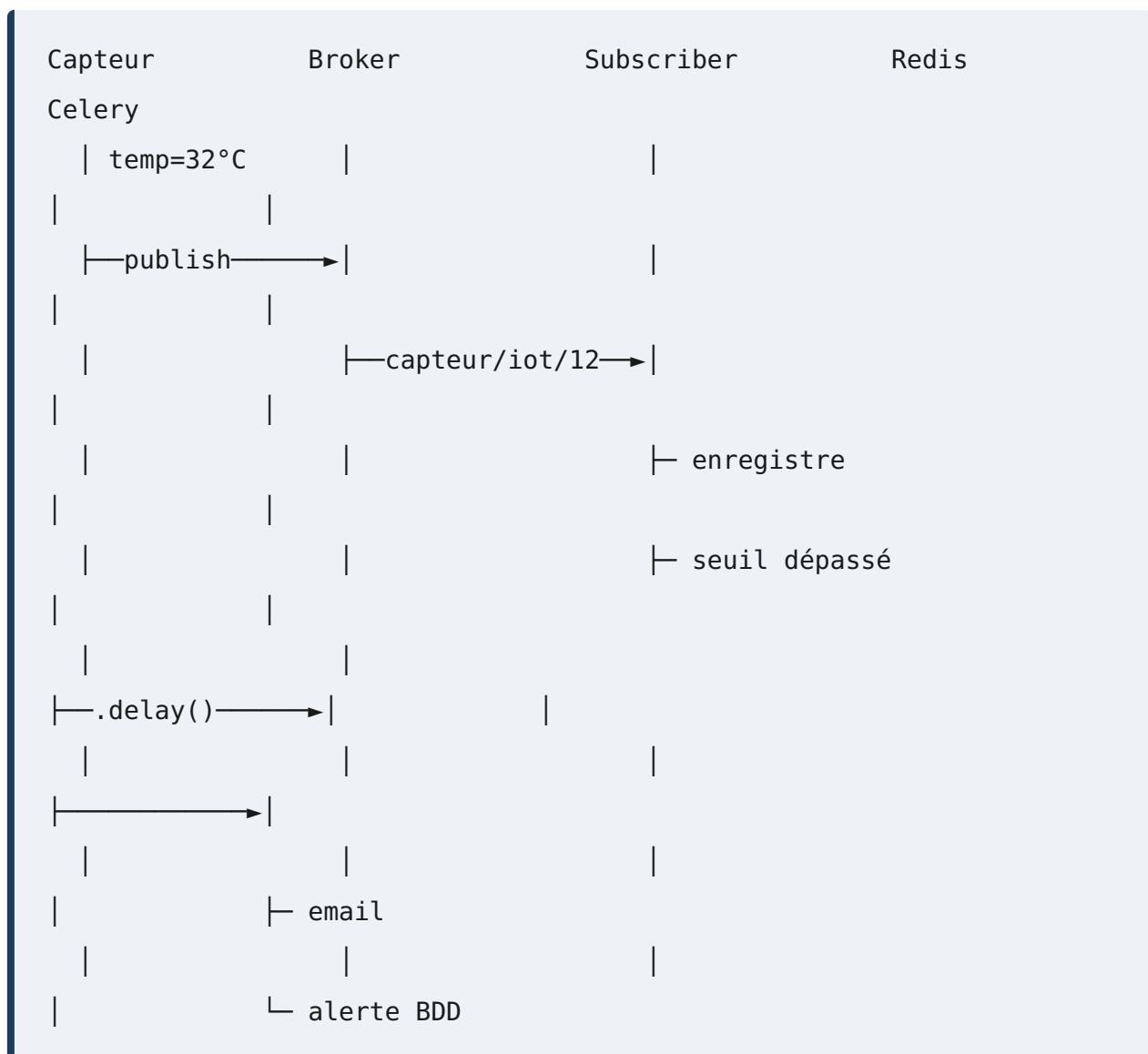
Pourquoi séparer MQTT de l'API web ?

Le subscriber tourne comme **processus indépendant**, pas dans le serveur web :

Raison	Explication
Boucle infinie	<code>loop_forever()</code> bloque — incompatible avec un serveur HTTP
Disponibilité	Les mesures continuent d'arriver même si le web est en maintenance
Isolation des pannes	Un plantage de l'API n'interrompt pas l'ingestion

C'est pour cela que le service tourne en plusieurs conteneurs (voir [Vue d'ensemble](#)).

Chaîne complète temps réel



Latence de bout en bout : quelques millisecondes côté subscriber, l'email partant en arrière-plan.

Lien avec les autres briques

- **Redis / Celery** : le subscriber ne fait que `.delay()` — voir [Celery & Redis](#).
- **Broker Mosquitto** : configuration, topics, sécurité TLS → livrable **Broker**.
- **Capteurs** : matériel, câblage, code Arduino → livrable **IoT**.

06 — Emails & notifications

Vue d'ensemble

Le backend exploitation envoie des emails HTML pour prévenir les responsables d'un dépassement de seuil, d'un capteur muet ou d'un incident déclaré. Tous ces envois passent par **Celery** — jamais en synchrone (voir [Celery & Redis](#)).

Types de notifications

Tâche Celery	Déclencheur	Contenu
<code>send_consolidated_alert_email</code>	Seuil température/humidité dépassé	Email groupé : lots affectés, valeur mesurée, seuil
<code>send_sensor_alert_email</code>	Capteur en anomalie	Nom du capteur, message, entrepôt
<code>send_incident_notification</code>	Incident déclaré manuellement	Description de l'incident
<code>check_stale_sensors</code>	Capteur muet (planifié, 15 s)	Alerte « capteur sans données »

Qui reçoit les emails ?

Les destinataires sont calculés dynamiquement à partir des **rôles** configurés dans les paramètres de l'exploitation.

```
_DEFAULT_NOTIFY_ROLES = ('responsable_exploitation',  
                           'responsable_qualite')
```

- Les rôles de notification sont stockés dans la table `Parameter` (`alert_notify_roles`, `incident_notify_roles`).
- La fonction `_get_recipient_emails()` récupère tous les utilisateurs **actifs** appartenant à ces groupes Django.

- Le rôle `admin` peut être inclus (superusers).

```
# Alertes → responsables exploitation + qualité
# Incidents → rôles configurés séparément
# Un utilisateur sans email est ignoré
```

Email consolidé

Plutôt qu'un email par mesure, le système **regroupe** : une seule notification liste tous les lots d'un entrepôt affectés par le même dépassement.

```
Objet : [ALERTE CRITIQUE] Température – Entrepôt Nord

Température mesurée : 32.4 °C (seuil : 28 °C)
Lots affectés :
  • Lot A-2026-014 (Arabica)
  • Lot A-2026-021 (Robusta)
```

Le HTML est généré côté Python (`_alert_html(...)`) avec un code couleur selon la gravité (critique / warning).

Anti-spam (cooldown)

Sans limite, un capteur oscillant autour du seuil enverrait des dizaines d'emails. Un cooldown en mémoire régule :

Gravité	Fréquence max par entrepôt
Critique	1 email / 30 min
Warning	1 email / 60 min

Configuration SMTP

```
# variables d'environnement
EMAIL_HOST      = smtp.gmail.com
EMAIL_PORT      = 587
EMAIL_HOST_USER = ...
```

```
EMAIL_HOST_PASSWORD = ... # mot de passe d'application
DEFAULT_FROM_EMAIL = noreply@futurekawa.com
```

Voir les identifiants de démonstration dans le livrable **Déploiement** et le fichier `.env.example`.

Résilience

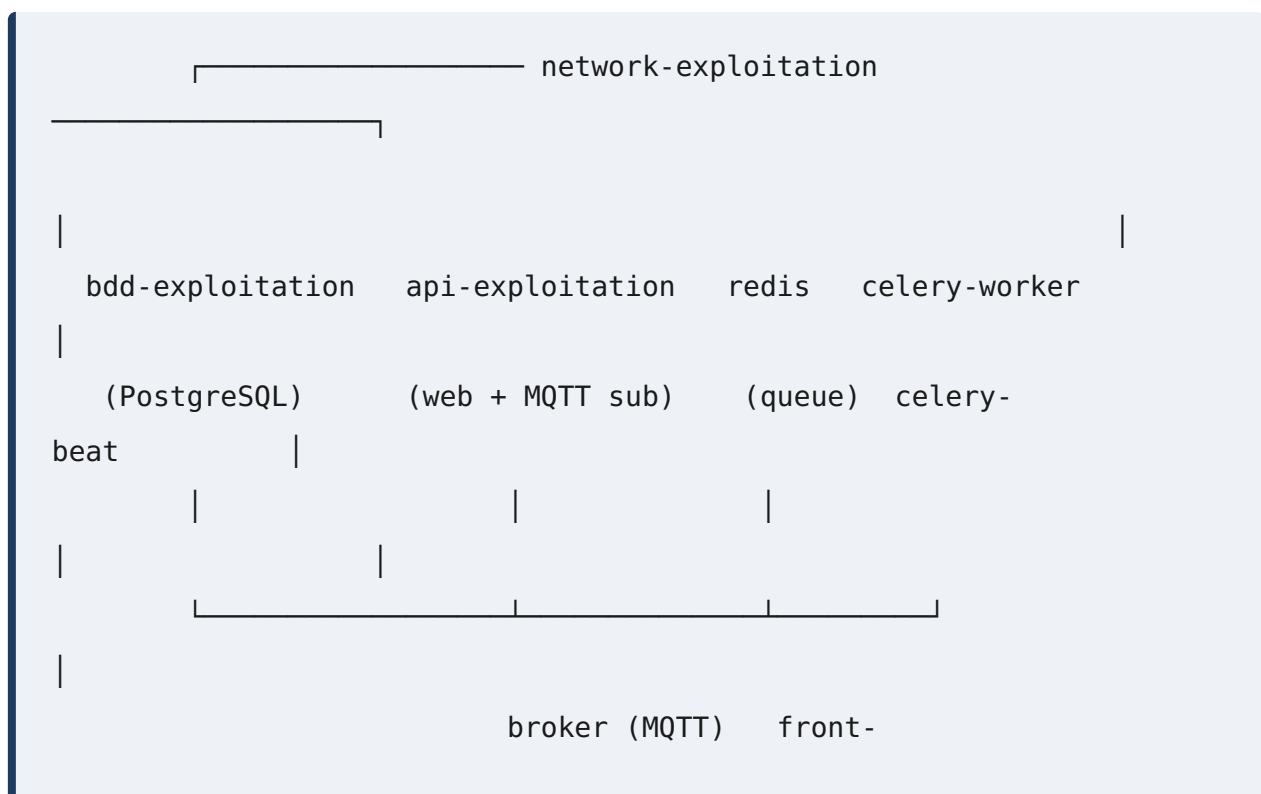
Toutes les tâches d'email sont déclarées `@shared_task(bind=True, max_retries=3)` : en cas d'échec SMTP, Celery réessaie 3 fois à 60 s d'intervalle avant d'abandonner. L'API et le subscriber ne sont jamais bloqués par un problème d'email.

07 — Configuration & déploiement

Les conteneurs du service

Le backend exploitation ne tourne pas dans un seul conteneur : chaque responsabilité a le sien (`docker-compose.exploit.yml`).

Conteneur	Image / commande	Rôle
<code>bdd-exploitation</code>	<code>postgres:16-alpine</code>	Base de données
<code>api-exploitation</code>	<code>/entrypoint.sh</code>	Migrations + subscriber MQTT + serveur Django
<code>redis</code>	<code>redis:7-alpine</code>	File de tâches Celery
<code>celery-worker</code>	<code>celery -A core worker -l info --concurrency=4</code>	Exécution des tâches async
<code>celery-beat</code>	<code>celery -A core beat -l info</code>	Tâches planifiées
<code>broker</code>	Mosquitto	Broker MQTT
<code>front-exploitation</code>	Nginx	Interface Vue.js



exploitation |

L'entrypoint

Au démarrage du conteneur `api-exploitation`, `entrypoint.sh` orchestre :

```
1. makemigrations + migrate          # met la base à jour
2. create_default_admin               # crée l'admin par défaut
3. Attend que le broker MQTT réponde  # nc -z broker 1883 (30 essais)
4. Lance mqtt_subscriber.py &        # subscriber en arrière-plan
5. exec runserver 0.0.0.0:5000        # serveur Django au premier
plan
```

Le worker et le beat Celery tournent dans **leurs propres conteneurs**, pas dans l'entrypoint : ils partagent le même code mais des commandes différentes.

Variables d'environnement clés

Variable	Rôle	Exemple
<code>DJANGO_SETTINGS_MODULE</code>	Environnement de settings	<code>core.settings.local</code>
<code>DATABASE_URL</code> / <code>POSTGRES_*</code>	Connexion PostgreSQL	—
<code>CELERY_BROKER_URL</code>	Redis broker	<code>redis://redis:6379/0</code>
<code>CELERY_RESULT_BACKEND</code>	Redis résultats	<code>redis://redis:6379/0</code>
<code>MQTT_BROKER</code> / <code>MQTT_PORT</code>	Broker MQTT	<code>broker</code> / <code>1883</code>
<code>EMAIL_HOST_USER</code> / <code>EMAIL_HOST_PASSWORD</code>	SMTP	—

Variable	Rôle	Exemple
MASTER_TOKEN	Accès siège → /api/all-data/	—

La liste complète est dans `.env.example` (racine du dépôt) et le livrable **Déploiement**.

Environnements de settings

```
core/settings/
├─ base.py          # commun
├─ local.py         # dev – DEBUG=True, CORS ouvert
├─ production.py    # prod – DEBUG=False, CORS restreint
└─ test.py          # CI – Celery synchrone, base SQLite
```

Démarrage local

```
# via Docker (recommandé)
docker compose -f docker-compose.exploit.yml up

# ou à la main, un terminal par processus :
python manage.py migrate
python manage.py runserver 0.0.0.0:5000      # web
python mqtt_subscriber.py                    # subscriber
celery -A core worker -l info                 # worker
celery -A core beat -l info                   # beat
```


08 — Tests

Approche

Les tests du backend exploitation s'appuient sur le framework de tests de Django (`TestCase` , base de test jetable) et sur les settings dédiés `core.settings.test` .

Particularité importante :

```
# core/settings/test.py
CELERY_TASK_ALWAYS_EAGER = True # les tâches Celery s'exécutent
immédiatement
CELERY_TASK_EAGER_PROPAGATES = True # les erreurs remontent
directement
DATABASES = { ... } # base de test isolée
```

En mode *EAGER* , **pas besoin de Redis ni de worker** : `.delay()` exécute la tâche sur-le-champ. Les tests restent rapides et déterministes.

Ce qui est couvert

Domaine	Exemples de tests
Endpoints CRUD	Création/lecture/mise à jour des lots, entrepôts, capteurs
Authentification	Login, token, set-password via invitation
Permissions	Accès par rôle / page
Tâches Celery	Création d'alerte, calcul des destinataires d'email

Domaine	Exemples de tests
Endpoint <code>/api/all-data/</code>	Réponse consolidée protégée par master token
Historiques & stats	<code>capteur/history</code> , <code>capteur/stats</code>

Lancer les tests

```
cd API-exploitation
DJANGO_SETTINGS_MODULE=core.settings.test python manage.py test

# ou via le Makefile / la CI
make test-exploitation
```

Intégration continue

Les tests backend sont rejoués dans la pipeline GitLab CI à chaque push (voir le livrable **CI/CD**). La couverture détaillée est documentée dans le livrable **Tests** (`03-couverture-api-exploitation.md`).